

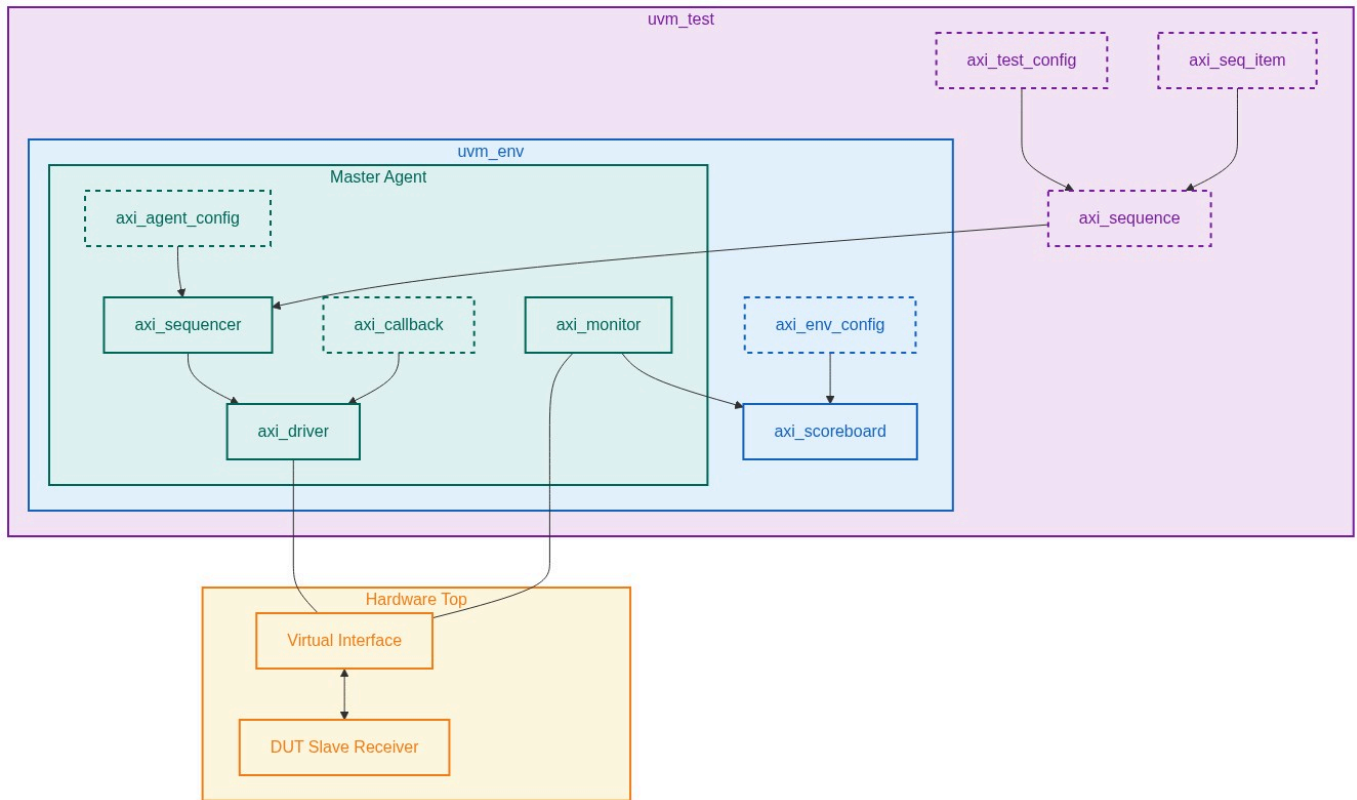
Verification Plan Report

Project: AXI-Stream Master VIP Verification Plan

Version: 19.0

Verification Role: Master

1. UVM Environment Architecture



2. Test Requirements

Index	Feature/Sub	Name	Description
REQ_MST_0 1	Handshake (TVALID/TREADY Handshake)	Valid Handshake Stability	Once TVALID is asserted, it must remain HIGH until the handshake occurs (TVALID & TREADY both HIGH). Transmitter must not wait for TREADY before asserting TVALID. Strategy: Drive TVALID and monitor for de-assertion without TREADY. Verify TDATA remains stable.
REQ_MST_0 2	Data Stream (Byte Qualifiers)	TKEEP/TSTRB Byte Selection	TKEEP determines if a byte is a data byte, position byte, or null byte. Strategy: Send streams with random TKEEP patterns (including null packets) and verify scoreboard data extraction matches.
REQ_MST_0 3	Packet Boundaries (TLAST Marking)	TLAST EOP Identification	TLAST marks the end of a packet. Strategy: Generate packets of variable sizes (1 beat to max) and verify TLAST is only HIGH on the last beat.

Verification Plan Report

REQ_MST_0 4	AXI5 Features (Wakeup Signaling)	TWAKEUP Triggering	TWAKEUP indicates activity and can trigger power-up. Strategy: Assert TWAKEUP and verify subsequent TVALID assertion. Test behavior of TWAKEUP de-assertion without activity.
REQ_MST_0 5	Data Integrity (Parity Protection)	AXI5 TDATACHK Parity	AXI5-Stream uses TDATACHK for parity. Strategy: Negative scenario: Inject 1-bit and 2-bit parity errors on TDATA and verify monitor flags an error.

3. Test Cases & Mapping

Index	Scenario	TR Mapping
TC_MST_001	Directed: TVALID asserted before TREADY. Verify TDATA stability.	REQ_MST_01
TC_MST_002	Directed: TREADY asserted before TVALID. Verify zero-wait-state transfer.	REQ_MST_01
TC_MST_003	Randomized: Multi-beat packet with variable TREADY stalls.	REQ_MST_01, REQ_MST_03
TC_MST_004	Randomized: Sparse stream with varying TKEEP bits (null and data bytes).	REQ_MST_02
TC_MST_005	Negative: De-assert TVALID before TREADY is seen HIGH.	REQ_MST_01
TC_MST_006	Directed: Minimum packet size (1 beat) with TLAST=1.	REQ_MST_03
TC_MST_007	Directed: TWAKEUP asserted 1 cycle before TVALID.	REQ_MST_04
TC_MST_008	Negative: Inject odd parity error on TDATACHK during active transfer.	REQ_MST_05

4. Functional Coverage Groups

Group Name	Description
cg_handshake_scenarios	Coverage of TVALID/TREADY relative timings.
cg_keep_patterns	Coverage of all possible TKEEP byte-lane combinations.
cg_packet_size	Coverage of packet sizes (single beat vs multi-beat).

5. Interface Signals

Signal Name	Direction	Width	Description
ACLK	Input	1	Global clock signal.
ARESETn	Input	1	Global active-LOW reset signal.
TVALID	Output	1	Indicates Transmitter is driving a valid transfer.
TREADY	Input	1	Indicates Receiver can accept a transfer.
TDATA	Output	`TDATA_WIDT H	Primary payload data channel.
TKEEP	Output	`TDATA_WIDT H/8	Qualifier for data bytes vs null bytes.

Verification Plan Report

TSTRB	Output	`TDATA_WIDT H/8	Qualifier for data bytes vs position bytes.
TLAST	Output	1	Packet boundary indicator.
TWAKEUP	Output	1	AXI5 wakeup indicator.

6. Verification Environment Structure

axi_stream_master_vip_tb/

```
|-- master_agent/  
|   |-- axi_stream_master_driver.sv  
|   |-- axi_stream_master_monitor.sv  
|   |-- axi_stream_master_agent_config.sv  
|   |-- axi_stream_master_callback.sv  
|   |-- axi_stream_master_sequencer.sv  
|-- env/  
|   |-- axi_stream_master_scoreboard.sv  
|   |-- axi_stream_master_env_config.sv  
|-- sequences/  
|   |-- axi_stream_master_base_sequence.sv  
|   |-- axi_stream_master_seq_item.sv  
|-- top/  
|   |-- axi_stream_master_tb_top.sv  
|   |-- axi_stream_master_test.sv  
|   |-- axi_stream_master_defines.sv
```

7. SystemVerilog Implementation

axi_stream_master_defines.sv (Architecture Blueprint)

```
COMPONENT: Global Defines  
PARAMETERS:  
- TDATA_WIDTH: 32 (Default payload width)  
- HAS_AXI5: 1 (Enables Wakeup/Parity)  
- ADDR_WIDTH: 32 (If applicable for interconnect addressing)  
LOGIC:  
- Shared parameters referenced by all UVM components.
```

axi_stream_master_seq_item.sv (Architecture Blueprint)

```
COMPONENT: axi_stream_master_seq_item  
EXTENDS: uvm_sequence_item  
MEMBERS:  
- rand logic [`TDATA_WIDTH-1:0] data  
- rand logic [`TDATA_WIDTH/8-1:0] keep  
- rand bit last  
- rand int inter_beat_delay  
LOGIC:
```

Verification Plan Report

- constraint c_delay { inter_beat_delay inside {[0:10]}; }
- constraint c_keep { keep != 0; } (Ensures at least one valid byte)

axi_stream_master_driver.sv (Architecture Blueprint)

```
COMPONENT: axi_stream_master_driver
EXTENDS: uvm_driver #(axi_stream_master_seq_item)
CONFIG_REF: axi_stream_master_agent_config (Source: ConfigDB)
MEMBERS:
  - virtual interface axi_stream_if vif
  - axi_stream_master_agent_config m_cfg
LOGIC:
  - build_phase: Retrieve m_cfg from DB; assign vif handle.
  - run_phase:
    - Wait for reset de-assertion.
    - loop:
      - get_next_item(req).
      - Drive TDATA, TKEEP, TLAST to vif pins.
      - Assert TVALID.
      - @(posedge vif.ACLK) while (TREADY is LOW).
      - item_done(req).
```

axi_stream_master_agent.sv (Architecture Blueprint)

```
COMPONENT: axi_stream_master_agent
EXTENDS: uvm_agent
CONFIG_REF: axi_stream_master_agent_config (Source: ConfigDB)
MEMBERS:
  - axi_stream_master_driver m_driver
  - axi_stream_master_monitor m_monitor
  - axi_stream_master_sequencer m_sqr
  - axi_stream_master_agent_config m_cfg
LOGIC:
  - build_phase:
    - Create m_monitor (unconditional).
    - if(m_cfg.is_active == UVM_ACTIVE): Create m_driver and m_sqr.
  - connect_phase:
    - if(m_cfg.is_active == UVM_ACTIVE): m_driver.seq_item_port.connect(m_sqr.seq_item_export).
```

axi_stream_master_test.sv (Architecture Blueprint)

```
COMPONENT: axi_stream_master_test
EXTENDS: uvm_test
MEMBERS:
  - axi_stream_master_env m_env
  - axi_stream_master_env_config m_env_cfg
  - axi_stream_master_agent_config m_agent_cfg
LOGIC:
  - build_phase:
```

Verification Plan Report

- Instantiate m_env_cfg and m_agent_cfg.
- Setup Virtual Interface in configs.
- uvm_config_db#(env_cfg)::set(this, "", "env_cfg", m_env_cfg).
- uvm_config_db#(agent_cfg)::set(this, "*agent*", "agent_cfg", m_agent_cfg).
- Create m_env.